

# Java 程序设计

---

CatMono

April 11, 2026

# 目 录

---

## I

## Java & Kotlin 核心基础

|          |                  |          |
|----------|------------------|----------|
| <b>1</b> | <b>Java 语言基础</b> | <b>6</b> |
| 1        | 占位内容 . . . . .   | 6        |
| <b>2</b> | <b>面向对象编程</b>    | <b>7</b> |
| 1        | 占位内容 . . . . .   | 7        |

## II

## JVM 标准库与常用工具

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>3</b> | <b>字符串处理</b>                             | <b>9</b>  |
| <b>4</b> | <b>集合框架</b>                              | <b>10</b> |
| 1        | 集合类概述 . . . . .                          | 10        |
| 2        | List 接口与实现 . . . . .                     | 10        |
| 2.1      | List 接口特点 . . . . .                      | 10        |
| 2.2      | ArrayList . . . . .                      | 12        |
| 2.3      | LinkedList . . . . .                     | 12        |
| 2.4      | Vector 与 Stack . . . . .                 | 13        |
| 2.5      | ArrayList 与 LinkedList 对比 . . . . .      | 13        |
| 3        | Set 接口与实现 . . . . .                      | 13        |
| 3.1      | HashSet . . . . .                        | 13        |
| 3.2      | LinkedHashSet . . . . .                  | 14        |
| 3.3      | TreeSet . . . . .                        | 14        |
| 3.4      | Set 的比较 . . . . .                        | 14        |
| 4        | Map 接口与实现 . . . . .                      | 15        |
| 4.1      | Map 接口常用方法 . . . . .                     | 15        |
| 4.2      | HashMap . . . . .                        | 15        |
| 4.3      | LinkedHashMap . . . . .                  | 17        |
| 4.4      | HashMap JDK 1.7 与 1.8 区别（面试重点） . . . . . | 18        |
| 4.5      | HashMap 扩容机制详解 . . . . .                 | 19        |

|      |                                                 |    |
|------|-------------------------------------------------|----|
| 5    | ConcurrentHashMap 原理（面试重点）                      | 19 |
| 5.1  | 为什么 HashMap 不是线程安全的？                            | 19 |
| 5.2  | ConcurrentHashMap 的分段锁（JDK 1.7）                 | 19 |
| 5.3  | ConcurrentHashMap 的 CAS + Synchronized（JDK 1.8） | 20 |
| 5.4  | TreeMap                                         | 21 |
| 5.5  | Hashtable、WeakHashMap 与 IdentityHashMap         | 21 |
| 5.6  | Map 的对比                                         | 21 |
| 6    | 集合工具类与实用方法                                      | 21 |
| 6.1  | Collections 工具类                                 | 21 |
| 6.2  | 常见实用方法                                          | 21 |
| 7    | 集合的遍历                                           | 22 |
| 7.1  | 迭代器核心接口                                         | 22 |
| 7.2  | Iterator                                        | 22 |
| 7.3  | ListIterator                                    | 23 |
| 7.4  | Splititerator                                   | 23 |
| 7.5  | 快速失败机制（fail-fast）                               | 24 |
| 7.6  | fail-fast 与 fail-safe（或弱一致）对比                   | 25 |
| 7.7  | 遍历与修改的安全规则                                      | 25 |
| 7.8  | 遍历方式对比                                          | 26 |
| 7.9  | 迭代器实战建议                                         | 26 |
| 8    | 多线程集合类                                          | 26 |
| 8.1  | Map 和 Set 并发实现                                  | 26 |
| 8.2  | 队列并发实现                                          | 27 |
| 8.3  | 并发集合与普通集合的选择                                    | 27 |
| 9    | 深拷贝                                             | 27 |
| 9.1  | 深拷贝与浅拷贝对比                                       | 27 |
| 9.2  | 方法一：构造函数复制                                      | 28 |
| 9.3  | 方法二：重写 clone() 方法                               | 28 |
| 9.4  | 方法三：Apache Commons Lang 序列化                     | 29 |
| 9.5  | 方法四：Gson 序列化                                    | 30 |
| 9.6  | 方法五：Jackson 序列化                                 | 30 |
| 9.7  | 深拷贝方法对比                                         | 31 |
| 10   | 补充说明                                            | 31 |
| 10.1 | 如何正确重写 equals 与 hashCode                        | 31 |
| 10.2 | 选择集合的经验法则                                       | 33 |

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| 10.3     | 时间复杂度参考                                     | 33        |
| 11       | 总结                                          | 33        |
| <b>5</b> | <b>异常处理</b>                                 | <b>34</b> |
| 1        | 异常概述                                        | 34        |
| 1.1      | 基本语法结构                                      | 34        |
| 1.2      | 非检查性异常 (Unchecked Exception)                | 34        |
| 1.3      | 检查性异常 (Checked Exception)                   | 35        |
| 2        | Throwable 类方法                               | 35        |
| 3        | 异常类型体系                                      | 36        |
| 3.1      | Error (系统错误)                                | 36        |
| 3.2      | 运行时异常 (RuntimeException)                    | 37        |
| 4        | 异常处理方式                                      | 37        |
| 4.1      | 声明异常 (throws)                               | 37        |
| 4.2      | 抛出异常 (throw)                                | 38        |
| 5        | try-catch-finally 详解                        | 38        |
| 5.1      | 各部分职责                                       | 39        |
| 5.2      | 多重 catch 块                                  | 39        |
| 6        | finally 执行时机                                | 39        |
| 6.1      | 基本执行规则                                      | 40        |
| 6.2      | return 与 finally 的交互                        | 40        |
| 6.3      | finally 不执行的特殊情况                            | 41        |
| 7        | 自定义异常                                       | 41        |
| 7.1      | 基本自定义异常                                     | 41        |
| 7.2      | 自定义异常的使用示例                                  | 41        |
| 8        | 异常处理最佳实践                                    | 42        |
| 8.1      | 异常处理原则                                      | 42        |
| 8.2      | 反模式示例                                       | 42        |
| 9        | 面试重点                                        | 43        |
| 9.1      | checked exception 与 unchecked exception 的区别 | 43        |
| 9.2      | try-catch-finally 执行顺序                      | 43        |
| 9.3      | throw 与 throws 的区别                          | 44        |
| 9.4      | finally 中 return 的问题                        | 44        |
| 9.5      | 异常处理与性能                                     | 45        |

|    |               |
|----|---------------|
| IV | 并发编程          |
| V  | 网络编程与高性能 IO   |
| VI | JVM 底层原理与性能调优 |

|   |                |    |
|---|----------------|----|
| 6 | JVM            | 50 |
| 1 | 占位内容 . . . . . | 50 |

# I

## Part

# Java & Kotlin 核心基础

|   |                 |   |
|---|-----------------|---|
| 1 | Java 语言基础 ..... | 6 |
| 1 | 占位内容 .....      | 6 |
| 2 | 面向对象编程 .....    | 7 |
| 1 | 占位内容 .....      | 7 |

# Chapter 1

## Java 语言基础

---

### 1 占位内容

这里是 chap01 的初始内容。

# Chapter 2

## 面向对象编程

---

### 1 占位内容

这里是 chap02 的初始内容。



# II

## Part

# JVM 标准库与常用工具

|    |                                     |    |
|----|-------------------------------------|----|
| 3  | 字符串处理 .....                         | 9  |
| 4  | 集合框架 .....                          | 10 |
| 1  | 集合类概述 .....                         | 10 |
| 2  | List 接口与实现 .....                    | 10 |
| 3  | Set 接口与实现 .....                     | 13 |
| 4  | Map 接口与实现 .....                     | 15 |
| 5  | ConcurrentHashMap 原理（面试重点）<br>..... | 19 |
| 6  | 集合工具类与实用方法 .....                    | 21 |
| 7  | 集合的遍历 .....                         | 22 |
| 8  | 多线程集合类 .....                        | 26 |
| 9  | 深拷贝 .....                           | 27 |
| 10 | 补充说明 .....                          | 31 |
| 11 | 总结 .....                            | 33 |
| 5  | 异常处理 .....                          | 34 |
| 1  | 异常概述 .....                          | 34 |
| 2  | Throwable 类方法 .....                 | 35 |
| 3  | 异常类型体系 .....                        | 36 |
| 4  | 异常处理方式 .....                        | 37 |
| 5  | try-catch-finally 详解 .....          | 38 |
| 6  | finally 执行时机 .....                  | 39 |
| 7  | 自定义异常 .....                         | 41 |
| 8  | 异常处理最佳实践 .....                      | 42 |
| 9  | 面试重点 .....                          | 43 |

# Chapter 3

## 字符串处理

---

# Chapter 4

## 集合框架

### 1 集合类概述

图片 (4.1) 展示了 Java 集合框架的整体结构，包括核心接口（Collection、List、Set、Map）以及主要实现类（ArrayList、LinkedList、HashSet、TreeSet、HashMap、TreeMap 等）。

Java 集合框架的设计目标主要有三个：**统一不同数据结构的使用方式**、**屏蔽底层实现差异**、**提供丰富且可组合的遍历和操作能力**。按照存储形式，可以将集合大致分为单列集合与双列集合两类：

- **单列集合**：以 Collection 为根接口，典型代表是 List 和 Set。
- **双列集合**：以 Map 为根接口，保存键值对映射关系。

常见实现类的定位可以概括为：

- ArrayList：动态数组，**随机访问快**，尾部增删高效。
- LinkedList：双向链表，**频繁插入和删除**更有优势。
- HashSet：基于哈希表的去重集合，通常无序。
- TreeSet：基于红黑树的有序集合，元素自动排序。
- HashMap：基于哈希表的键值对容器，**查询效率高**。
- TreeMap：基于红黑树的有序键值对容器，**适合范围查询**。

#### NOTE

List 与 Set 的区别：

- **有序性**：List 保证按插入顺序排序，Set 存储和取出顺序不一致。
- **唯一性**：List 元素可以重复，Set 元素唯一。
- **获取元素**：List 可以通过索引直接操作元素，Set 不可以。

### 2 List 接口与实现

#### 2.1 List 接口特点

List 是**有序集合**，允许重复元素，并保留元素的插入顺序。它最重要的特征是**支持通过索引访问元素**，因此适合“按位置读写”的场景。

List 接口常用方法包括：

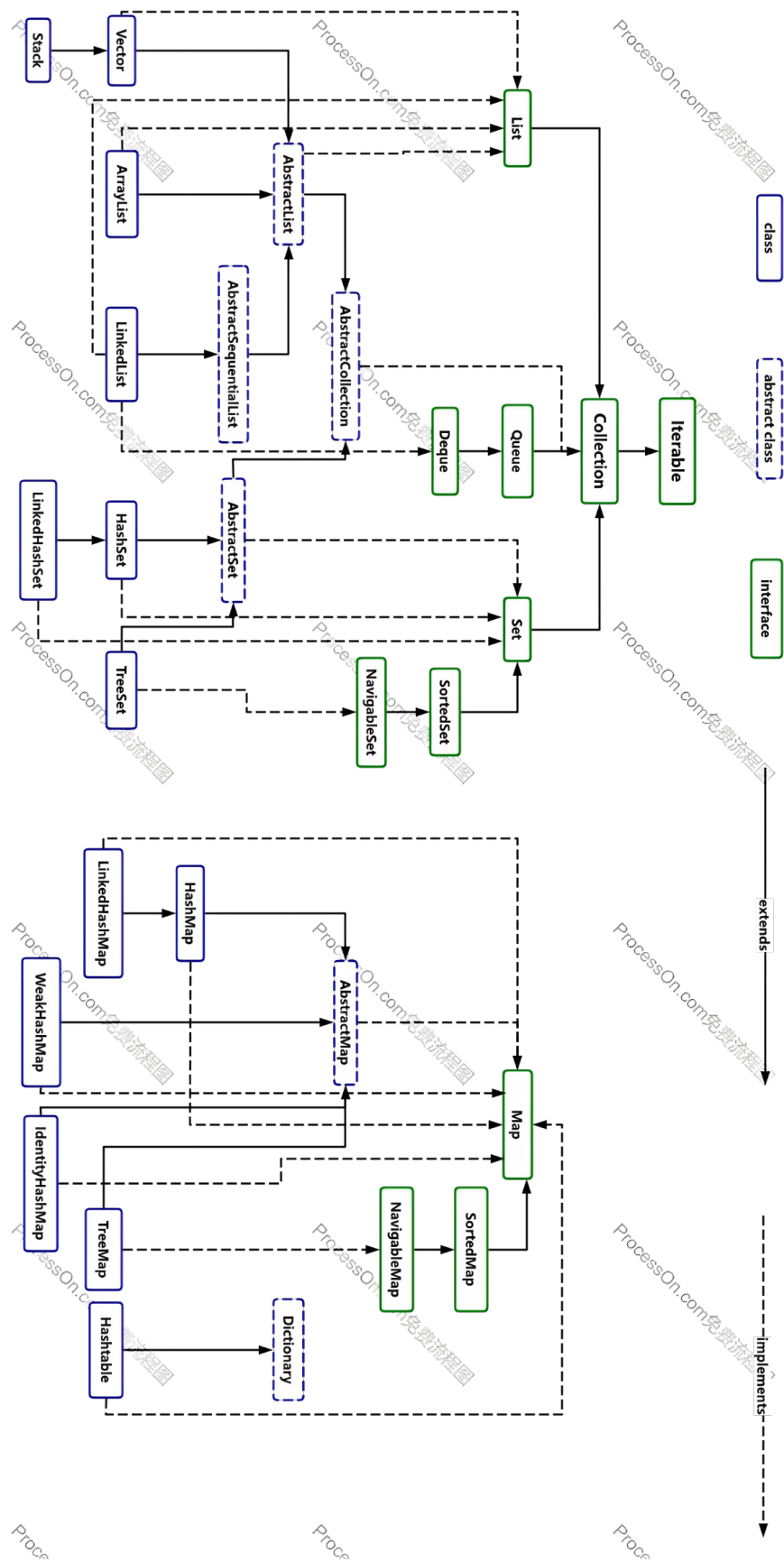


Figure 4.1: Java 集合框架概览

| 方法                                     | 描述   | 说明       |
|----------------------------------------|------|----------|
| <code>add(E e)</code>                  | 添加元素 | 默认追加到末尾  |
| <code>get(int index)</code>            | 获取元素 | 按索引读取    |
| <code>set(int index, E e)</code>       | 修改元素 | 替换指定位置的值 |
| <code>remove(int index)</code>         | 删除元素 | 按索引删除    |
| <code>subList(int from, int to)</code> | 子列表  | 左闭右开     |

## Collection 接口方法汇总

Collection 是 List、Set 的父接口，定义了集合的基本操作：

| 方法                              | 描述       | 注意事项                     |
|---------------------------------|----------|--------------------------|
| <code>add(E e)</code>           | 添加元素     | 返回是否成功                   |
| <code>remove(Object o)</code>   | 移除元素     | 移除首次出现的元素                |
| <code>clear()</code>            | 清空集合     |                          |
| <code>isEmpty()</code>          | 判断是否为空   |                          |
| <code>size()</code>             | 返回元素个数   |                          |
| <code>contains(Object o)</code> | 判断是否包含元素 | 依赖 <code>equals()</code> |
| <code>iterator()</code>         | 获取迭代器    | 用于遍历                     |
| <code>toArray()</code>          | 转换为数组    |                          |

## 2.2 ArrayList

ArrayList 是最常用的 List 实现，底层基于**动态数组**。它的优势是**随机访问速度快**，适合频繁读取、偶尔修改的场景。

- **优点**：按索引访问效率高，尾插性能好，内存连续，缓存友好。
- **缺点**：中间插入和删除可能需要移动大量元素。
- **适用场景**：读取远多于修改，例如结果缓存、列表展示、批量存储。

ArrayList 的扩容并不是每次只加 1，而是按比例扩容。这样做的目的是降低频繁扩容带来的**拷贝成本**。

ArrayListExample.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 List<String> names = new ArrayList<>();
5 names.add("Alice");
6 names.add("Bob");
7 names.add(1, "Tom");
8
9 System.out.println(names.get(0));
10 System.out.println(names);

```

## 2.3 LinkedList

LinkedList 的底层是**双向链表**。它在频繁插入、删除，尤其是头尾操作时更有优势。

- **优点**：中间插入和删除时，不需要像数组那样整体移动大量元素。
- **缺点**：随机访问慢，需要从头或尾逐步查找。

- **适用场景**：频繁在头部、中部、尾部插入和删除的场景。

LinkedList 还实现了 Deque 接口，因此可用于**队列**和**双端队列**。

LinkedListExample.java

```
1 import java.util.LinkedList;
2
3 LinkedList<Integer> queue = new LinkedList<>();
4 queue.addLast(10);
5 queue.addLast(20);
6 queue.addFirst(5);
7
8 System.out.println(queue.removeFirst());
9 System.out.println(queue);
```

## 2.4 Vector 与 Stack

Vector 是较早的动态数组实现，方法大多带同步控制，因此**线程安全**，但性能通常不如 ArrayList。Stack 则是基于 Vector 的栈实现，遵循**后进先出（LIFO）**原则。

现在更推荐直接使用 ArrayDeque 作为栈结构的替代，因为它更轻量、效率更高。

## 2.5 ArrayList 与 LinkedList 对比

| 对比项  | ArrayList           | LinkedList              |
|------|---------------------|-------------------------|
| 底层结构 | 动态数组                | 双向链表                    |
| 随机访问 | <b>快</b> ( $O(1)$ ) | <b>慢</b> ( $O(n)$ )     |
| 插入删除 | 中间较慢，需要移动元素         | 频繁修改更合适，只需修改指针          |
| 内存特性 | 连续空间，缓存友好           | 节点额外开销较大 (prev/next 指针) |
| 典型场景 | 读多写少、随机访问多          | 频繁增删、头尾操作多              |
| 内存占用 | 低（仅数组开销）            | 高（每个节点需存储指针）            |
| 线程安全 | 否                   | 否（但方法同步较多）              |

### NOTE

以下情况使用 ArrayList（本质是数组，顺序存储）：

- 频繁访问列表中的某一个元素。
- 只需要在列表末尾进行添加和删除元素操作。

以下情况使用 LinkedList（本质是链表，链式存储）：

- 频繁在列表中间插入和删除元素。
- 需要支持高效的头尾操作。

虽然 LinkedList 名字中有“List”，但它实际上实现了 Deque 接口，可以当作队列、栈或双端队列使用。

## 3 Set 接口与实现

Set 表示**不允许重复元素**的集合。它没有索引概念，因此不能像 List 那样通过位置直接获取元素。对于是否“重复”的判断，核心依赖元素的 equals() 和 hashCode() 方法。

### 3.1 HashSet

HashSet 基于**哈希表**实现，元素的存放位置由哈希值决定，因此通常不保证顺序。

- **优点：**查找、插入、删除通常接近  $O(1)$ 。
- **缺点：**无序，且对哈希函数质量比较敏感。
- **适用场景：**去重、成员判断、元素唯一性维护。

### 3.2 LinkedHashSet

LinkedHashSet 在 HashSet 的基础上维护了元素的插入顺序。它适合需要去重，同时又希望保留插入顺序的场景。

### 3.3 TreeSet

TreeSet 基于**红黑树**实现，能够对元素自动排序。默认按照自然顺序排列，也可以通过比较器自定义顺序。

TreeSet 常用的有序操作包括：

| 方法                   | 作用            |
|----------------------|---------------|
| first()              | 返回第一个元素       |
| last()               | 返回最后一个元素      |
| pollFirst()          | 取出并删除第一个元素    |
| pollLast()           | 取出并删除最后一个元素   |
| headSet(E to)        | 返回小于给定元素的子集   |
| subSet(E from, E to) | 返回指定范围的子集     |
| tailSet(E from)      | 返回大于等于给定元素的子集 |

### 3.4 Set 的比较

| 实现类           | 有序    | 排序方式     | 允许重复 | 典型用途   | 查找效率        | null 支持 |
|---------------|-------|----------|------|--------|-------------|---------|
| HashSet       | 否     | -        | 否    | 快速去重   | $O(1)$      | 是       |
| LinkedHashSet | 按插入顺序 | -        | 否    | 去重并保序  | $O(1)$      | 是       |
| TreeSet       | 是     | 自然顺序/比较器 | 否    | 排序集合   | $O(\log n)$ | 否       |
| EnumSet       | 是     | 枚举顺序     | 否    | 枚举类型去重 | $O(1)$      | 否       |

#### NOTE

HashSet 与 TreeSet 的相同点：

- 它们都不允许存储重复元素，如果传入重复元素只会记录第一个。
- 它们都无法通过索引去获取或修改元素。

不同点：

- 底层数据结构不同，分别是哈希表和树。
- TreeSet 有序排列，HashSet 不一定有序。
- HashSet 查找效率更高 ( $O(1)$ )，TreeSet 需要比较 ( $O(\log n)$ )。

## 4 Map 接口与实现

Map 用于保存键值对关系，键唯一，值可以重复。相比 List 和 Set，Map 的核心是通过 key 快速定位 value。

### 4.1 Map 接口常用方法

| 方法                   | 作用         |
|----------------------|------------|
| put(key, value)      | 添加或修改键值对   |
| get(key)             | 根据键获取值     |
| remove(key)          | 删除指定键对应的映射 |
| containsKey(key)     | 判断是否存在指定键  |
| containsValue(value) | 判断是否存在指定值  |
| keySet()             | 获取所有键的集合视图 |
| values()             | 获取所有值的集合视图 |
| entrySet()           | 获取键值对集合视图  |

### 4.2 HashMap

HashMap 是最常用的 Map 实现。它基于**哈希表**，允许 null 键和 null 值，且通常不保证顺序。

HashMap 的核心特性可以概括为：

- 通过哈希函数定位桶位置，平均查询效率较高。
- JDK 8 之后，在链表过长时可能**树化为红黑树**，以提升极端情况下的性能。
- **不是线程安全的**，多线程并发修改时应优先使用 ConcurrentHashMap。

HashMap 的底层结构可以理解为“数组 + 链表 + 红黑树”。当多个键发生哈希冲突时，它们会落到同一个桶里；如果冲突过多，链表可能转为红黑树，以避免退化成线性查找。

HashMapExample.java

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 Map<String, Integer> scores = new HashMap<>();
5 scores.put("Java", 95);
6 scores.put("Python", 92);
7 scores.put(null, 100);
8
9 System.out.println(scores.get("Java"));

```

HashMap 需要特别关注两个点：

- equals() 与 hashCode() 必须保持一致性，否则可能出现“放得进去、取不出来”的问题。
- 如果作为键的对象是自定义类，通常需要正确重写这两个方法。

### HashMap 底层数据结构详解

HashMap 的底层是**数组 + 链表 + 红黑树**的复合结构：

- **数组（桶）**：Node<K,V>[] table，是 HashMap 的主体，每个数组元素称为一个“桶”（bucket），用于存储键值对。



- **链表**：当多个键的哈希值相同，导致它们被映射到同一个桶时，这些键值对会以链表的形式存储在该桶中。
- **红黑树**：当某个桶中的链表长度过长时，为了避免  $O(n)$  的线性查找性能瓶颈，链表会转化为红黑树，将查找时间复杂度优化到  $O(\log n)$ 。

## HashMap 关键参数

| 参数                   | 默认值  | 含义     | 说明                 |
|----------------------|------|--------|--------------------|
| initialCapacity      | 16   | 初始容量   | 数组的初始长度，必须是 2 的幂次方 |
| loadFactor           | 0.75 | 负载因子   | 衡量 HashMap 填充程度的阈值 |
| TREEIFY_THRESHOLD    | 8    | 树化阈值   | 链表长度达到此值时尝试转化为红黑树  |
| UNTREEIFY_THRESHOLD  | 6    | 退化阈值   | 红黑树节点数减少到此值时退化为链表  |
| MIN_TREEIFY_CAPACITY | 64   | 最小树化容量 | 只有数组容量达到此值时才会进行树化  |

### NOTE

负载因子 0.75 是时间和空间成本的平衡。负载因子过高会减少空间开销，但增加哈希碰撞的概率；过低则会浪费空间。

## 树化条件

只有同时满足以下两个条件，HashMap 才会将某个桶中的链表转化为红黑树：

1. **链表长度**  $\geq 8$ ：该桶中的链表节点数达到了 TREEIFY\_THRESHOLD（默认 8）。
2. **数组容量**  $\geq 64$ ：整个 HashMap 的数组长度达到了 MIN\_TREEIFY\_CAPACITY（默认 64）。

如果只满足第一个条件，但数组容量不足 64，HashMap 会优先选择扩容（resize），而不是树化。当红黑树节点数减少到 UNTREEIFY\_THRESHOLD（默认 6）时，为了节省空间和维护成本，红黑树会退化为普通链表。

### INFO

#### 为什么选择红黑树而不是其他平衡二叉树？

- 红黑树是**近似平衡**的二叉查找树，插入和删除操作的旋转次数较少，性能更优。
- 红黑树的查找、插入、删除时间复杂度都是  $O(\log n)$ ，且常数因子较小。
- 相比 AVL 树（严格平衡），红黑树对插入和删除更友好，适合频繁修改的场景。

## Hash 函数

为了将键均匀地分布到数组的各个桶中，HashMap 使用了精心设计的哈希函数：

### hash 方法

```

1 static final int hash(Object key) {
2     int h;
3     return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
4 }
```

其工作原理如下：

1. 首先获取 key.hashCode() 的值。
2. 然后将哈希值的高 16 位与低 16 位进行异或 (^)，这一步称为“扰动”，目的是让哈希值的高位也参与到后续的下标计算中，减少哈希碰撞的概率。

- 最后通过  $(n - 1) \& \text{hash}$  计算出数组下标 ( $n$  是数组容量, 必须是 2 的幂)。

## | put() 方法执行流程

当调用 `put(key, value)` 方法时, `HashMap` 会执行以下步骤:

- 计算哈希值:** 调用 `hash(key)` 方法计算出键的哈希值。
- 定位桶:** 通过  $(n - 1) \& \text{hash}$  计算出应该存储在数组中的哪个桶 (下标)。
- 处理冲突:**
  - 如果该桶为空, 直接将新的节点放入该位置。
  - 如果该桶不为空, 说明发生了哈希碰撞, 需要遍历链表:
    - 如果找到一个节点的键与 `key` 相等 (通过 `equals()` 方法比较), 则更新该节点的值并返回旧值。
    - 如果遍历到链表末尾仍未找到相等的键, 则将新节点添加到链表末尾 (**尾插法**, JDK 8 以前使用头插法)。
- 检查链表长度:** 如果添加新节点后链表长度达到 `TREEIFY_THRESHOLD` (默认 8), 则尝试将链表转化为红黑树 (前提是数组容量达到 `MIN_TREEIFY_CAPACITY`, 否则优先扩容)。

## | get() 方法执行流程

当调用 `get(key)` 方法时, `HashMap` 会执行以下步骤:

- 计算哈希值:** 调用 `hash(key)` 方法计算出键的哈希值。
- 定位桶:** 通过  $(n - 1) \& \text{hash}$  计算出应该查找的桶 (下标)。
- 查找元素:**
  - 如果该桶为空, 直接返回 `null`。
  - 如果该桶不为空, 说明可能存在哈希碰撞, 需要遍历链表:
    - 如果找到一个节点的键与 `key` 相等 (通过 `equals()` 方法比较), 则返回该节点的值。
    - 如果遍历到链表末尾仍未找到相等的键, 则返回 `null`。

## | 扩容机制

- 触发条件:** 当 `size > capacity * loadFactor` 时触发。
- 扩容过程:** 创建一个新的数组, 容量是原数组的 2 倍, 然后将原数组中的所有节点重新哈希到新数组中。
- 优化:** JDK 8 中, 扩容时会根据节点的哈希值与旧容量的按位与结果, 将节点分为两部分, 分别放到新数组的原下标和原下标 + 旧容量的位置, 避免了重新计算哈希, 提升了效率。

### INFO

#### 为什么 `HashMap` 的容量必须是 2 的幂次方?

- 为了使用位运算  $(n - 1) \& \text{hash}$  代替 `hash % n`, 提高计算效率。
- 当  $n$  为 2 的幂次方时,  $n - 1$  的二进制全是 1, 保证了下标计算的均匀性。

## 4.3 LinkedHashMap

`LinkedHashMap` 是 `HashMap` 的子类, 它在哈希表的基础上维护了双向链表, 因此既能快速查找, 又能保留顺序。

它支持两种顺序模式:

- 插入顺序：默认行为，元素按进入顺序排列。
- 访问顺序：每次访问元素后，元素会被移动到末尾，常用于 LRU 缓存。

LinkedHashMap 之所以能继承 HashMap，是因为 HashMap 预留了钩子方法：afterNodeAccess、afterNodeInsertion、afterNodeRemoval。HashMap 自己空实现，LinkedHashMap 重写它们来维护双向链表的结构，从而实现了元素的有序性。

LRUCacheExample.java

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 Map<Integer, String> cache = new LinkedHashMap<>(4, 0.75f, true) {
5     @Override
6     protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
7         return size() > 3;
8     }
9 };
10
11 cache.put(1, "A");
12 cache.put(2, "B");
13 cache.put(3, "C");
14 cache.get(1);
15 cache.put(4, "D");
16 System.out.println(cache);
17 // 输出: {2=B, 3=C, 4=D} 键 1 被淘汰了
```

关于 **LRU (Least Recently Used, 最近最少使用)** 缓存淘汰算法，LinkedHashMap 提供了一个非常方便的实现方式。通过重写 `removeEldestEntry(Map.Entry<K,V> eldest)` 方法，可以在每次插入新元素时自动检查是否需要淘汰最旧的元素。

#### WARNING

使用 LRU 缓存时，需要将 `accessOrder` 设置为 `true`，这样每次访问（`get` 或 `put`）都会将该元素移动到链表末尾。

## 手动实现 LRU 缓存

另一种实现方式是手动实现：手写一个双向链表来维护访问顺序，HashMap 来存储键值对和链表节点的映射关系。每次访问或插入元素时，将对应节点移动到链表头部，当缓存超过容量时，移除链表尾部的节点。

## 4.4 HashMap JDK 1.7 与 1.8 区别（面试重点）

| 区别点  | JDK 1.7 | JDK 1.8       |
|------|---------|---------------|
| 底层结构 | 数组 + 链表 | 数组 + 链表 + 红黑树 |
| 插入方式 | 头插法     | 尾插法           |
| 扩容计算 | 重新计算哈希  | 原位置或原位置 + 旧容量 |
| 线程安全 | 否（非同步）  | 否（非同步）        |

## 为什么 JDK 8 改成尾插法？

JDK 1.7 使用**头插法**，在并发扩容时会导致**循环链表**，从而引发死循环。JDK 1.8 改成**尾插法**，避免了环形链表的产生，但 HashMap 本身仍然是**非线程安全**的。

**WARNING****HashMap 多线程并发问题:**

- JDK 1.7 并发扩容时, 使用头插法可能导致**死循环** (环形链表)。
- JDK 1.8 虽然解决了死循环问题, 但仍然存在**数据丢失**的风险。
- 多线程环境下应使用 ConcurrentHashMap。

HashMap 死循环示意.java

```
1 // JDK 1.7 并发扩容可能导致死循环
2 // 线程 A 和线程 B 同时扩容
3 // 线程 A 进行 rehash 时被挂起
4 // 线程 B 完成 rehash, 创建环形链表
5 // 后续调用 get() 时可能陷入死循环
```

**CAUTION**

如果在面试中被问到 HashMap, 一定要主动说明:

1. HashMap 是**非线程安全**的。
2. JDK 1.7 在并发扩容时可能产生**死循环**。
3. JDK 1.8 解决了死循环问题, 但仍不是线程安全的。
4. 多线程环境应使用 ConcurrentHashMap 或 Collections.synchronizedMap()。

## 4.5 HashMap 扩容机制详解

HashMap 的扩容过程是一个 **rehashing** 的过程:

1. **触发条件:** 当  $\text{size} > \text{capacity} * \text{loadFactor}$  时触发 (默认:  $\text{size} > 16 * 0.75 = 12$ )。
2. **扩容过程:**
  - 创建新数组, 容量为原来的 2 倍。
  - 将所有元素重新计算位置并迁移到新数组。
3. **JDK 1.8 优化:** 扩容时不再重新计算哈希, 而是利用  $(e.\text{hash} \& \text{oldCap}) == 0$  来判断元素去向。

**NOTE**

扩容是**昂贵的操作**, 涉及内存分配和元素迁移。如果预知数据量, 应使用**带初始容量**的构造函数, 避免多次扩容: `new HashMap<>(initialCapacity)` 或 `new HashMap<>(expectedSize, 0.75f)`。

## 5 ConcurrentHashMap 原理 (面试重点)

### 5.1 为什么 HashMap 不是线程安全的?

- **数据覆盖:** 多线程并发 put 时, 可能导致数据覆盖。
- **死循环** (JDK 1.7): 并发扩容时形成环形链表, 导致 get() 死循环。
- **putIfAbsent 问题:** 复合操作不是原子性的。

### 5.2 ConcurrentHashMap 的分段锁 (JDK 1.7)

JDK 1.7 的 ConcurrentHashMap 使用 **Segment 分段锁**技术:

- 内部包含一个 Segment 数组, 每个 Segment 持有一把锁。
- 不同 Segment 之间的操作可以**并行进行**。

- 锁的粒度是 Segment，而非整个 HashMap。

## 5.3 ConcurrentHashMap 的 CAS + Synchronized (JDK 1.8)

JDK 1.8 之后，ConcurrentHashMap 放弃了分段锁，转而使用：

- **CAS (Compare-And-Swap)**：用于**无锁**的读操作和部分写操作。
- **synchronized**：用于**加锁**的写操作，锁的粒度细化到单个桶。

这种方式比分段锁更高效，因为：

- 读操作大多使用 CAS，不需要加锁。
- 写操作只锁定当前桶，其他桶仍可并发访问。
- 锁的粒度更细，并发度更高。

| 特性   | JDK 1.7                     | JDK 1.8            |
|------|-----------------------------|--------------------|
| 锁类型  | ReentrantLock (Segment 分段锁) | synchronized + CAS |
| 锁粒度  | Segment (多个桶)               | 单个桶 (Node)         |
| 并发度  | 16 (Segment 数组长度)           | 更高 (每个桶独立)         |
| 数据结构 | 数组 + 链表                     | 数组 + 链表 + 红黑树      |

**面试高频问题：**

- ConcurrentHashMap 如何保证线程安全？
- JDK 1.7 和 1.8 在实现上有什么区别？
- 为什么 ConcurrentHashMap 的 get() 不需要加锁？
- ConcurrentHashMap 和 Hashtable、Collections.synchronizedMap() 的区别？

LRUCacheManual.java

```

1 public class LRUCache<K, V> {
2     private final int capacity;
3     private final Map<K, Node<K, V>> map;
4     private final DoubleLinkedList<K, V> list;
5
6     public LRUCache(int capacity) {
7         this.capacity = capacity;
8         this.map = new HashMap<>();
9         this.list = new DoubleLinkedList<>();
10    }
11
12    public V get(K key) {
13        if (!map.containsKey(key)) return null;
14        Node<K, V> node = map.get(key);
15        list.moveToHead(node);
16        return node.value;
17    }
18
19    public void put(K key, V value) {
20        if (map.containsKey(key)) {
21            Node<K, V> node = map.get(key);
22            node.value = value;
23            list.moveToHead(node);
24        } else {
25            if (map.size() >= capacity) {
26                Node<K, V> tail = list.removeTail();
27                map.remove(tail.key);
28            }
29            Node<K, V> newNode = new Node<>(key, value);
30            list.addToHead(newNode);
31            map.put(key, newNode);
32        }
33    }
34 }

```

## 5.4 TreeMap

TreeMap 基于**红黑树**实现，键会按照自然顺序或比较器顺序排列。它适合范围查询、排序输出等有序场景。

TreeMap 的注意事项：

- **键不能为空**，因为需要参与比较。
- 查询效率通常为  $O(\log n)$ ，低于 HashMap 的平均  $O(1)$ 。
- 当需要“按顺序”访问键时，TreeMap 很合适。

## 5.5 Hashtable、WeakHashMap 与 IdentityHashMap

Hashtable 是较早的线程安全 Map 实现，整体风格偏旧，现代代码中通常更推荐 ConcurrentHashMap。

WeakHashMap 的键使用**弱引用**，当外部没有强引用指向这些键时，键值对可能被垃圾回收，适合做缓存或附属数据结构。

IdentityHashMap 的键比较不是依赖 equals()，而是依赖**引用相等 (==)**。这使得它适用于需要“对象身份”而不是“对象内容”判断的场景。

## 5.6 Map 的对比

| 实现类           | 底层结构           | 线程安全 | 有序 | null 支持         |
|---------------|----------------|------|----|-----------------|
| HashMap       | 数组 + 链表 + 红黑树  | 否    | 否  | key 和 value 都允许 |
| LinkedHashMap | HashMap + 双向链表 | 否    | 是  | 允许              |
| Hashtable     | 数组 + 链表        | 是    | 否  | 不允许             |
| TreeMap       | 红黑树            | 否    | 是  | key 不允许         |

## 6 集合工具类与实用方法

Java 提供了 Collections、Arrays 和一些工厂方法，帮助我们更方便地处理集合。

### 6.1 Collections 工具类

Collections 是集合工具类，提供了大量静态方法：

- sort(List)：排序。
- reverse(List)：反转顺序。
- shuffle(List)：打乱顺序。
- binarySearch(List, key)：二分查找。
- unmodifiableList(Set/Map)：创建不可修改视图。
- synchronizedList(Set/Map)：创建同步包装器。

### 6.2 常见实用方法

- List.of(...), Set.of(...), Map.of(...)：创建不可变集合。
- new ArrayList<>(collection)：按现有集合构造新集合。
- Arrays.asList(...)：数组转列表视图，注意其大小固定。



- `removeIf(predicate)`: 按条件删除元素。

CollectionsExample.java

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.List;
4
5 List<Integer> numbers = new ArrayList<>(List.of(3, 1, 4, 2));
6 Collections.sort(numbers);
7 Collections.reverse(numbers);
8 System.out.println(numbers);

```

## 7 集合的遍历

集合遍历是集合框架中最基础也最容易出错的部分。遍历时是否允许修改、如何修改、会不会抛异常，是学习这一部分最重要的三个问题。

### 7.1 迭代器核心接口

Java 中所有集合（Collection 及其子类）都实现了 `Iterable<E>` 接口，该接口提供了 `iterator()` 方法，用于返回一个 `Iterator<E>` 对象。

| 接口                                 | 所在包                    | 主要能力        | 典型场景                 | 线程安全          |
|------------------------------------|------------------------|-------------|----------------------|---------------|
| <code>Iterator&lt;E&gt;</code>     | <code>java.util</code> | 单向遍历、删除当前元素 | 所有 Collection 的基础遍历  | 否 (fail-fast) |
| <code>ListIterator&lt;E&gt;</code> | <code>java.util</code> | 双向遍历、按位置增删改 | List 专用，需前后移动或边遍历边编辑 | 否             |
| <code>Spliterator&lt;T&gt;</code>  | <code>java.util</code> | 可分割遍历、并行友好  | Stream 并行处理、批量遍历优化   | 否             |

### 7.2 Iterator

`Iterator` 是 Collection 的通用遍历方式。它的核心方法是 `hasNext()`、`next()` 和 `remove()`。

| 方法                                      | 作用                                | 注意事项                                                              |
|-----------------------------------------|-----------------------------------|-------------------------------------------------------------------|
| <code>boolean hasNext()</code>          | 判断是否还有下一个元素可遍历                    | 不移动迭代器指针                                                          |
| <code>E next()</code>                   | 返回下一个元素，并将迭代器指针向后移动一位             | 若没有元素，抛出 <code>NoSuchElementException</code>                      |
| <code>void remove()</code>              | 删除上一次 <code>next()</code> 方法返回的元素 | 1. 必须在 <code>next()</code> 之后调用；2. 每次 <code>next()</code> 后只能调用一次 |
| <code>forEachRemaining(Consumer)</code> | 对迭代器剩余的元素执行给定操作                   | JDK 8 引入，适合批量处理剩余元素                                               |

IteratorExample.java

```

1 import java.util.ArrayList;
2 import java.util.Iterator;
3 import java.util.List;
4
5 List<String> list = new ArrayList<>(List.of("A", "B", "C"));
6 Iterator<String> it = list.iterator();
7
8 while (it.hasNext()) {
9     String value = it.next();
10    if ("B".equals(value)) {
11        it.remove();
12    }
13 }
14
15 System.out.println(list); // [A, C]

```

需要注意的是，`remove()` 只能在一次 `next()` 之后调用一次，连续调用会抛出异常。

**WARNING**

在同一轮遍历中，`remove()` 只能在一次 `next()` 后调用一次；连续调用会抛 `IllegalStateException`。

## 7.3 ListIterator

`ListIterator` 只适用于 `List`，支持**正向和反向遍历**，也支持在遍历中就地修改。

| 方法                                         | 说明                                      |
|--------------------------------------------|-----------------------------------------|
| <code>hasPrevious() / previous()</code>    | 向前遍历                                    |
| <code>nextIndex() / previousIndex()</code> | 获取当前位置索引                                |
| <code>set(E e)</code>                      | 替换最近一次 <code>next/previous</code> 返回的元素 |
| <code>add(E e)</code>                      | 在当前位置插入元素                               |
| <code>remove()</code>                      | 删除最近一次 <code>next/previous</code> 返回的元素 |

ListIteratorExample.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3 import java.util.ListIterator;
4
5 List<String> list = new ArrayList<>(List.of("a", "b", "c"));
6 ListIterator<String> lit = list.listIterator();
7
8 while (lit.hasNext()) {
9     String s = lit.next();
10    if ("b".equals(s)) {
11        lit.set("B"); // 替换
12        lit.add("b2"); // 在当前位置后插入
13    }
14 }
15
16 while (lit.hasPrevious()) {
17     System.out.print(lit.previous() + " "); // 反向遍历
18 }
19 // 输出示例: c b2 B a

```

## 7.4 Spliterator

`Spliterator` 的目标是**把一个遍历任务拆分成多个子任务**，以支持并行处理。它是 `Stream` 的底层重要支撑之一。

| 方法                                      | 说明                                 |
|-----------------------------------------|------------------------------------|
| <code>tryAdvance(Consumer)</code>       | 消费一个元素，若存在返回 <code>true</code>     |
| <code>forEachRemaining(Consumer)</code> | 消费剩余全部元素                           |
| <code>trySplit()</code>                 | 尝试拆分，返回新的 <code>Spliterator</code> |
| <code>estimateSize()</code>             | 估算剩余元素个数                           |
| <code>characteristics()</code>          | 返回特征位（有序、可并行等）                     |

常见特征值：



| 特征         | 含义        |
|------------|-----------|
| ORDERED    | 元素有固定遍历顺序 |
| SIZED      | 大小已知      |
| SORTED     | 元素已排序     |
| DISTINCT   | 元素唯一      |
| NONNULL    | 元素非 null  |
| IMMUTABLE  | 数据源不可变    |
| CONCURRENT | 支持并发修改    |

SpliteratorExample.java

```

1 List<Integer> nums = IntStream.rangeClosed(1, 10).boxed().toList();
2 Spliterator<Integer> sp1 = nums.spliterator();
3 Spliterator<Integer> sp2 = sp1.trySplit();
4
5 if (sp2 != null) {
6     sp2.forEachRemaining(n -> System.out.print("L" + n + " "));
7 }
8 sp1.forEachRemaining(n -> System.out.print("R" + n + " "));

```

## 7.5 快速失败机制 (fail-fast)

大多数 java.util 普通集合迭代器（如 ArrayList、HashMap）采用**快速失败**机制。

原理如下：

1. 每个集合内部维护一个 modCount（修改计数），记录集合结构被修改的次数（如 add()、remove()）。
2. 每个迭代器创建时会记录当前的 modCount 作为 expectedModCount。
3. 每次调用 next() 或 remove() 时，迭代器会检查当前集合的 modCount 是否与 expectedModCount 相等。

如果不等，说明集合在迭代过程中被外部修改，立即抛出 ConcurrentModificationException，而不是继续执行不确定的逻辑。

FailFastExample.java

```

1 List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
2
3 for (Integer x : list) {
4     if (x == 2) {
5         list.remove(x); // 可能触发 ConcurrentModificationException
6     }
7 }

```

### NOTE

快速失败是“尽力检测”机制，不保证 100% 每次都在同一位置抛出。它主要用于尽早暴露并发/误用问题，不是线程安全机制。

为避免快速失败：

- 遍历过程中，必须使用迭代器自身的 remove()/add() 方法修改集合。
- 对于并发场景，可以使用 java.util.concurrent 包下的线程安全集合，它们的迭代器不依赖 modCount，不会触发快速失败。

**CAUTION**

**面试重点：ConcurrentModificationException 常见场景：**

1. 在 for-each 循环中使用集合自身的 remove() 方法。
2. 多线程环境下，一个线程遍历集合，另一个线程修改集合。
3. 单线程环境下，快速连续修改集合（如循环删除多个元素）。

## 7.6 fail-fast 与 fail-safe（或弱一致）对比

| 类型              | 代表容器                                   | 遍历时修改集合 | 特点           | 数据一致性 |
|-----------------|----------------------------------------|---------|--------------|-------|
| fail-fast       | ArrayList、HashMap                      | 抛异常     | 检测修改并快速失败    | 最终一致  |
| fail-safe / 弱一致 | CopyOnWriteArrayList、ConcurrentHashMap | 通常不抛异常  | 迭代基于快照或弱一致视图 | 弱一致   |

## 7.7 遍历与修改的安全规则

- 删除当前元素：优先使用 Iterator.remove()。
- 需要前后移动和插入修改：使用 ListIterator。
- 条件删除：优先使用 removeIf()。
- 多线程场景：优先使用并发集合，例如 CopyOnWriteArrayList 或 ConcurrentHashMap。

常用安全遍历修改方法：

SafeModificationExample.java

```

1 // 方法一：使用 Iterator.remove()
2 List<String> list1 = new ArrayList<>(List.of("A", "B", "C"));
3 Iterator<String> it = list1.iterator();
4 while (it.hasNext()) {
5     String s = it.next();
6     if ("B".equals(s)) {
7         it.remove(); // 安全删除
8     }
9 }
10
11 // 方法二：使用 removeIf()
12 List<String> list2 = new ArrayList<>(List.of("A", "B", "C"));
13 list2.removeIf(s -> "B".equals(s)); // 安全删除满足条件的元素
14
15 // 方法三：倒序遍历
16 List<String> list3 = new ArrayList<>(List.of("A", "B", "C"));
17 for (int i = list3.size() - 1; i >= 0; i--) {
18     if ("B".equals(list3.get(i))) {
19         list3.remove(i); // 安全删除
20     }
21 }

```

MapTraversalExample.java

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 Map<String, Integer> map = new HashMap<>();
5 map.put("A", 1);
6 map.put("B", 2);
7
8 for (Map.Entry<String, Integer> entry : map.entrySet()) {
9     if ("B".equals(entry.getKey())) {
10         entry.setValue(20); // 安全修改值
11     }
12 }

```

```
12 }
```

记忆一句话：遍历中要改集合，就让“迭代器自己改”；需要并发，再换并发容器。

7.8 遍历方式对比

| 遍历方式         | 底层实现          | 优点         | 缺点               | 适用场景       |
|--------------|---------------|------------|------------------|------------|
| 普通 for       | 基于索引          | 随机访问效率高    | 对 LinkedList 效率低 | 需要频繁随机访问   |
| 增强 for       | Iterator      | 语法简洁       | 无法遍历中修改          | 简单遍历       |
| Iterator     | 迭代器模式         | 支持遍历中安全删除  | 代码繁琐             | 需要动态修改集合   |
| ListIterator | 迭代器模式         | 支持双向、就地增删改 | 仅适用 List         | 需要前后移动编辑   |
| Stream API   | Splititerator | 函数式、支持并行   | 无法直接修改原集合        | 数据过滤、映射、统计 |

7.9 迭代器实战建议

- 1. 只读遍历：可用 for-each 或 forEach。
- 2. 遍历中删除：优先 Iterator.remove() 或 removeIf。
- 3. 需要前后移动与就地编辑：使用 ListIterator。
- 4. 大数据并行处理：结合 Splititerator 与 parallelStream。
- 5. 多线程并发遍历：优先并发容器，避免在普通集合上边遍历边改。

8 多线程集合类

java.util.concurrent 包提供了许多线程安全或更适合并发场景的集合类。它们并不一定“全面更慢”，而是针对并发读写做了更适合的设计。

8.1 Map 和 Set 并发实现

| 集合类                   | 底层实现                  | 特点                 | 使用场景       |
|-----------------------|-----------------------|--------------------|------------|
| ConcurrentHashMap     | 哈希表 + 链表 + 红黑树，CAS    | 高效并发访问，键值都不能为 null | 高并发键值存储、缓存 |
| CopyOnWriteArrayList  | 动态数组，写时复制             | 读无锁，写复制数组          | 读多写少、配置缓存  |
| CopyOnWriteArraySet   | CopyOnWriteArrayList  | 元素不重复，写性能低         | 线程安全去重     |
| ConcurrentSkipListMap | 跳表                    | 元素有序排序             | 有序键值对、范围查询 |
| ConcurrentSkipListSet | ConcurrentSkipListMap | 元素有序排序             | 有序任务集合     |

## 8.2 队列并发实现

| 集合类                   | 底层实现   | 特点               | 使用场景          |
|-----------------------|--------|------------------|---------------|
| LinkedBlockingQueue   | 链表阻塞队列 | FIFO, 可指定容量      | 生产者-消费者、线程池任务 |
| ArrayBlockingQueue    | 数组阻塞队列 | 必须指定容量, 性能高      | 高效线程安全队列      |
| PriorityBlockingQueue | 堆无界队列  | 按优先级排序, 可能OOM    | 优先级任务调度       |
| DelayQueue            | 优先级队列  | 元素需实现 Delayed 接口 | 延时消息、定时器      |
| SynchronousQueue      | 无存储空间  | 插入必须等待取出         | 线程间直接传递数据     |
| LinkedTransferQueue   | 链表无界队列 | 支持直接 transfer 数据 | 高效数据传递        |
| BlockingDeque         | 双向链表   | 两端插入删除, 可用做栈/队列  | 任务回退、双向分发     |

并发集合的一个重要特点是：它们往往不会像普通集合那样在遍历时快速失败，而是采用快照或弱一致性策略，保证在并发环境中“能用、稳定、可预期”。

### WARNING

虽然并发集合支持在多线程环境下使用, 但并不意味着所有操作都是原子的。例如 ConcurrentHashMap 的 size()、isEmpty() 等方法在并发场景下可能返回近似值, 精确操作仍需加锁。

## 8.3 并发集合与普通集合的选择

| 场景        | 普通集合               | 并发集合                    | 原因         |
|-----------|--------------------|-------------------------|------------|
| 单线程环境     | ArrayList, HashMap | -                       | 并发集合有额外开销  |
| 高并发读多写少   | ArrayList, HashMap | CopyOnWriteArrayList    | 读操作无锁, 开销小 |
| 高并发读写均衡   | -                  | ConcurrentHashMap       | 桶级别锁, 性能好  |
| 需要有序遍历    | TreeMap, TreeSet   | ConcurrentSkipListMap   | 跳表支持有序并发访问 |
| 生产者-消费者   | ArrayList          | BlockingQueue 系列        | 阻塞机制简化并发控制 |
| 需要 LRU 缓存 | LinkedHashMap      | ConcurrentHashMap + 自定义 | 需要额外实现淘汰策略 |

# 9 深拷贝

Java 中对象拷贝通常分为**浅拷贝**和**深拷贝**。浅拷贝只复制对象引用，深拷贝则会把对象及其内部引用对象都复制一份。

- **浅拷贝**：外层对象变了，内层引用仍然共享。
- **深拷贝**：外层和内层对象都独立，互不影响。

深拷贝的常用实现方式包括：[构造函数复制](#)、[重写 clone\(\)](#)、[序列化复制](#)、[JSON 反序列化复制](#)等。

## 9.1 深拷贝与浅拷贝对比

| 类型  | 实现方式                | 优缺点           | 适用场景       |
|-----|---------------------|---------------|------------|
| 浅拷贝 | Object.clone() 默认行为 | 简单高效, 但共享内部引用 | 对象不含可变引用成员 |
| 深拷贝 | 递归拷贝所有引用类型          | 完全独立, 但实现复杂   | 复杂对象图的完整复制 |

UserAndAddress.java

```
1 // 用户类
2 public class User {
3     private String name;
4     private Address address;
5     // constructors, getters and setters
6 }
7
8 // 地址类
9 public class Address {
10    private String city;
11    private String country;
12    // constructors, getters and setters
13 }
```

## 9.2 方法一：构造函数复制

最直接的方式是在构造函数里手动复制每一层对象。优点是简单明确，缺点是对象层级多时代码较繁琐。

ConstructorCopy.java

```
1 @Test
2 public void constructorCopy() {
3     Address address = new Address("杭州", "中国");
4     User user = new User("大山", address);
5     // 调用构造函数时进行深拷贝
6     User copyUser = new User(user.getName(),
7                               new Address(address.getCity(), address.getCountry()));
8     // 修改源对象的值
9     user.getAddress().setCity("深圳");
10    // 检查两个对象的值不同
11    assertNotSame(user.getAddress().getCity(),
12                  copyUser.getAddress().getCity());
13 }
```

## 9.3 方法二：重写 clone() 方法

Object 父类有个 clone() 的拷贝方法，不过它是 protected 类型的，我们需要重写它并修改为 public 类型。除此之外，子类还需要实现 Cloneable 接口来告诉 JVM 这个类是可以拷贝的。

### NOTE

super.clone() 其实是浅拷贝，所以在重写类的 clone() 方法时，所有引用类型的成员变量都需要调用其 clone() 方法重新赋值。

CloneableClasses.java

```
1 // 地址类，实现 Cloneable 接口
2 public class Address implements Cloneable {
3     private String city;
4     private String country;
5
6     @Override
7     public Address clone() throws CloneNotSupportedException {
8         return (Address) super.clone();
9     }
10 }
11
12 // 用户类，实现 Cloneable 接口
13 public class User implements Cloneable {
14     private String name;
15     private Address address;
16 }
```

```

17  @Override
18  public User clone() throws CloneNotSupportedException {
19      User user = (User) super.clone();
20      user.setAddress(this.address.clone()); // 关键：深拷贝
21      return user;
22  }
23  }

```

CloneCopyTest.java

```

1  @Test
2  public void cloneCopy() throws CloneNotSupportedException {
3      Address address = new Address("杭州", "中国");
4      User user = new User("大山", address);
5
6      // 调用 clone() 方法进行深拷贝
7      User copyUser = user.clone();
8
9      // 修改源对象的值
10     user.getAddress().setCity("深圳");
11
12     // 检查两个对象的值不同
13     assertNotSame(user.getAddress().getCity(),
14         copyUser.getAddress().getCity());
15 }

```

## 9.4 方法三：Apache Commons Lang 序列化

Java 提供了序列化的能力，我们可以先将源对象进行序列化，再反序列化生成拷贝对象。但是，使用序列化的前提是拷贝的类（包括其成员变量）需要实现 `Serializable` 接口。Apache Commons Lang 包对 Java 序列化进行了封装，我们可以直接使用它。

SerializableClasses.java

```

1  // 地址类，实现 Serializable 接口
2  public class Address implements Serializable {
3      private String city;
4      private String country;
5  }
6
7  // 用户类，实现 Serializable 接口
8  public class User implements Serializable {
9      private String name;
10     private Address address;
11 }

```

SerializationUtilsClone.java

```

1  @Test
2  public void serializableCopy() {
3      Address address = new Address("杭州", "中国");
4      User user = new User("大山", address);
5
6      // 使用 Apache Commons Lang 序列化进行深拷贝
7      User copyUser = (User) SerializationUtils.clone(user);
8
9      // 修改源对象的值
10     user.getAddress().setCity("深圳");
11
12     // 检查两个对象的值不同
13     assertNotSame(user.getAddress().getCity(),
14         copyUser.getAddress().getCity());
15 }

```

## 9.5 方法四: Gson 序列化

Gson 可以将对象序列化成 JSON, 也可以将 JSON 反序列化成对象, 所以我们可以用它进行深拷贝。

GsonCopy.java

```
1 @Test
2 public void gsonCopy() {
3     Address address = new Address("杭州", "中国");
4     User user = new User("大山", address);
5
6     // 使用 Gson 序列化进行深拷贝
7     Gson gson = new Gson();
8     User copyUser = gson.fromJson(gson.toJson(user), User.class);
9
10    // 修改源对象的值
11    user.getAddress().setCity("深圳");
12
13    // 检查两个对象的值不同
14    assertNotSame(user.getAddress().getCity(),
15                  copyUser.getAddress().getCity());
16 }
```

## 9.6 方法五: Jackson 序列化

Jackson 与 Gson 相似, 可以将对象序列化成 JSON, 明显不同的地方是拷贝的类 (包括其成员变量) 需要有默认的空参构造函数。

JacksonClasses.java

```
1 // 用户类, 需要默认空参构造函数
2 public class User {
3     private String name;
4     private Address address;
5
6     public User() {} // 默认空参构造函数
7
8     public User(String name, Address address) {
9         this.name = name;
10        this.address = address;
11    }
12 }
13
14 // 地址类, 需要默认空参构造函数
15 public class Address {
16     private String city;
17     private String country;
18
19     public Address() {} // 默认空参构造函数
20
21     public Address(String city, String country) {
22         this.city = city;
23         this.country = country;
24     }
25 }
```

JacksonCopy.java

```
1 @Test
2 public void jacksonCopy() throws IOException {
3     Address address = new Address("杭州", "中国");
4     User user = new User("大山", address);
5
6     // 使用 Jackson 序列化进行深拷贝
7     ObjectMapper objectMapper = new ObjectMapper();
8     User copyUser = objectMapper.readValue(
9         objectMapper.writeValueAsString(user), User.class);
10 }
```



```
10
11 // 修改源对象的值
12 user.getAddress().setCity("深圳");
13
14 // 检查两个对象的值不同
15 assertNotSame(user.getAddress().getCity(),
16               copyUser.getAddress().getCity());
17 }
```

## 9.7 深拷贝方法对比

| 方法           | 依赖库              | 优点         | 缺点                      | 适用场景             |
|--------------|------------------|------------|-------------------------|------------------|
| 构造函数         | 无                | 简单直接，类型安全  | 代码繁琐，层级深时难以维护           | 小型简单对象           |
| clone()      | 无                | JDK 原生支持   | 需实现 Cloneable，引用字段需递归处理 | 中等复杂度对象          |
| Commons Lang | commons-lang3    | 封装良好，使用简单  | 需实现 Serializable        | 复杂对象图            |
| Gson         | Gson             | 跨语言友好，格式可读 | 性能一般                    | JSON interchange |
| Jackson      | jackson-databind | 性能好，功能丰富   | 需无参构造函数                 | 高性能 JSON 处理      |

实际开发中，如果对象结构复杂且需要长期维护，优先考虑显式拷贝或专门的映射层，而不是把深拷贝完全交给序列化工具。

## 10 补充说明

### 10.1 如何正确重写 equals 与 hashCode

当对象会被放入 HashSet、HashMap 之类依赖哈希结构的集合时，equals() 与 hashCode() 必须一起重写。一般原则是：如果两个对象通过 equals() 判定相等，那么它们的 hashCode() 也必须相等。

#### equals()

判断两个对象是否相等，默认实现是比较引用地址。

#### hashCode()

返回对象的哈希值，用于哈希表快速定位。

正确重写 equals() 需要遵循以下五个原则：

1. **自反性**：x.equals(x) 必须返回 true。
2. **对称性**：x.equals(y) 和 y.equals(x) 结果必须一致。
3. **传递性**：若 x.equals(y) 和 y.equals(z) 都为 true，则 x.equals(z) 必须为 true。
4. **一致性**：在对象未被修改的情况下，多次调用 equals() 结果必须一致。
5. **非空性**：x.equals(null) 必须返回 false。

equals() 与 hashCode() 的契约关系：

- 如果两个对象相等（equals() 返回 true），则它们的 hashCode() 必须相等。
- 如果两个对象的 hashCode() 相等，它们不一定相等（可能发生哈希碰撞）。



**NOTE**

如果重写了 `equals()` 但不重写 `hashCode()`，会导致“放得进去、取不出来”的问题。例如，将对象放入 `HashSet` 后，`contains()` 可能返回 `false`。

常见的问题是使用 `equals()` 判断相等但用 `==` 判断哈希相等，或反之。正确做法是始终同时重写这两个方法。

**WARNING**

对于 `String`、`Integer` 等 JDK 类，它们的 `equals()` 和 `hashCode()` 已经正确实现，不需要再重写。

PersonClass.java

```
1 // 示例：正确重写 equals() 和 hashCode()
2 public class Person {
3     private String name;
4     private int age;
5
6     public Person(String name, int age) {
7         this.name = name;
8         this.age = age;
9     }
10
11     @Override
12     public boolean equals(Object o) {
13         if (this == o) return true; // 自反性
14         if (o == null || getClass() != o.getClass()) return false;
15         Person person = (Person) o;
16         return age == person.age &&
17             Objects.equals(name, person.name);
18     }
19
20     @Override
21     public int hashCode() {
22         return Objects.hash(name, age); // 使用 Objects.hash()
23     }
24 }
```

在 IDE（如 IntelliJ IDEA）中，可以使用快捷键自动生成 `equals()` 和 `hashCode()` 方法，确保正确性。

现代 Java（15+）可以使用 `record` 类型自动获得正确实现：

PersonRecord.java

```
1 // 使用 record 简化 equals/hashCode 实现
2 public record Person(String name, int age) {}
```

使用 Lombok 的项目也可以使用注解：

PersonLombok.java

```
1 import lombok.EqualsAndHashCode;
2
3 @EqualsAndHashCode
4 public class Person {
5     private String name;
6     private int age;
7 }
```

## 10.2 选择集合的经验法则

| 需求      | 首选                   | 备选            | 特点          | 有序    | 线程安全 | null 支持   |
|---------|----------------------|---------------|-------------|-------|------|-----------|
| 按下标访问   | ArrayList            | -             | 随机访问 $O(1)$ | 按插入顺序 | 否    | 是         |
| 频繁头尾插删  | ArrayDeque           | LinkedList    | 两端操作高效      | 否     | 否    | 是         |
| 去重      | HashSet              | LinkedHashSet | 查找 $O(1)$   | 否/按插入 | 否    | 是 (value) |
| 保持插入顺序  | LinkedHashSet        | -             | 去重 + 保序     | 按插入顺序 | 否    | 是 (value) |
| 自动排序    | TreeSet              | -             | 红黑树实现       | 按比较器  | 否    | 否 (key)   |
| 键值对存储   | HashMap              | -             | 查找 $O(1)$   | 否     | 否    | 是         |
| 有序键值对   | TreeMap              | LinkedHashMap | 按比较器排序      | 按比较器  | 否    | key 否     |
| 高并发键值存储 | ConcurrentHashMap    | -             | 桶级别锁        | 否     | 是    | 否         |
| 读多写少    | CopyOnWriteArrayList | -             | 写时复制        | 按插入顺序 | 是    | 是         |
| 生产者-消费者 | BlockingQueue        | -             | 阻塞机制        | FIFO  | 是    | 否         |

集合框架的学习重点不只是“记住类名”，而是理解每个容器背后的数据结构、访问模式、是否有序、是否允许重复以及是否线程安全。只要掌握了这些维度，绝大多数集合题和工程场景都能快速做出正确选择。

## 10.3 时间复杂度参考

| 操作    | ArrayList | LinkedList | HashSet | TreeSet     | HashMap | TreeMap     |
|-------|-----------|------------|---------|-------------|---------|-------------|
| 添加    | $O(1)$    | $O(1)$     | $O(1)$  | $O(\log n)$ | $O(1)$  | $O(\log n)$ |
| 删除    | $O(n)$    | $O(1)$     | $O(1)$  | $O(\log n)$ | $O(1)$  | $O(\log n)$ |
| 查找    | $O(1)$    | $O(n)$     | $O(1)$  | $O(\log n)$ | $O(1)$  | $O(\log n)$ |
| 按索引访问 | $O(1)$    | $O(n)$     | -       | -           | -       | -           |

### NOTE

ArrayList 和 LinkedList 的时间复杂度差异主要体现在：

- ArrayList 按索引访问是  $O(1)$ ，但中间插入/删除需要移动元素，最坏情况是  $O(n)$ 。
- LinkedList 插入/删除本身是  $O(1)$ ，但需要先定位到目标位置，最坏是  $O(n)$ 。

## 11 总结

Java 集合框架是一套设计精良的 API，其核心价值在于：

1. **统一接口**：通过接口抽象，让不同数据结构的集合拥有统一的操作方式。
2. **封装实现**：使用者无需关心底层是数组、链表还是树，只需要选择合适的集合类型。
3. **丰富工具**：Collections、Arrays 工具类以及 Stream API 提供了强大的操作能力。
4. **并发支持**：java.util.concurrent 包提供了丰富的线程安全集合。

掌握集合框架的关键是理解：

- 每个集合的底层数据结构
- 时间空间复杂度特性
- 是否有序、是否允许重复、是否线程安全
- 适用场景与选择依据

# Chapter 5

## 异常处理

Java 中的异常处理机制与 C# 基本一致，但具有更完善的异常体系结构。本章将详细介绍 Java 异常的分类、处理方式以及最佳实践。

### 1 异常概述

#### 1.1 基本语法结构

Java 提供了 **try-catch-finally** 结构来捕获和处理异常：

ExceptionDemo.java

```
1 try {  
2     // 需要进行异常捕获的代码块  
3     // 可能抛出异常的代码放在这里  
4 } catch (Exception ex) {  
5     // 捕获异常信息，catch 可以存在多个，用于捕获不同异常  
6     ex.printStackTrace();  
7 } finally {  
8     // 不管前面的代码块是否异常，finally 中的内容都会执行  
9     // 通常用于资源释放，如关闭流、连接等  
10 }
```

#### 1.2 非检查性异常（Unchecked Exception）

非检查性异常是 **RuntimeException** 及其子类，编译器不会强制检查。这类异常通常由程序逻辑错误引起。

| 异常类                             | 描述                               |
|---------------------------------|----------------------------------|
| ArithmeticException             | 当出现异常的运算条件时抛出，例如整数“除以零”          |
| ArrayIndexOutOfBoundsException  | 用非法索引访问数组时抛出，索引为负或大于等于数组大小       |
| ArrayStoreException             | 试图将错误类型的对象存储到对象数组时抛出             |
| ClassCastException              | 试图将对象强制转换为不是实例的子类时抛出             |
| IllegalArgumentException        | 向方法传递了不合法或不正确的参数时抛出              |
| IllegalMonitorStateException    | 线程试图等待对象的监视器但本身没有指定监视器           |
| IllegalStateException           | 在非法或不适当的时间调用方法时产生信号              |
| IllegalThreadStateException     | 线程没有处于请求操作所要求的适当状态时抛出            |
| IndexOutOfBoundsException       | 排序索引超出范围时抛出（数组、字符串或向量）           |
| NegativeArraySizeException      | 应用程序试图创建大小为负的数组时抛出               |
| NullPointerException            | 应用程序试图在需要对象的地方使用 <b>null</b> 时抛出 |
| NumberFormatException           | 字符串不能转换为适当数值格式时抛出                |
| SecurityException               | 由安全管理器抛出，表示存在安全侵犯                |
| StringIndexOutOfBoundsException | String 方法中索引为负或超出字符串大小           |
| UnsupportedOperationException   | 当不支持请求的操作时抛出                     |

**NOTE**

**NullPointerException** 是 Java 开发中最常见的异常之一，通常发生在：

- 调用 null 对象的属性或方法
- 访问 null 数组的元素
- 集合中添加 null 后再进行操作

## 1.3 检查性异常（Checked Exception）

检查性异常是 **Exception** 的子类（除 **RuntimeException** 外），编译器会强制检查，必须进行处理。

| 异常类                        | 描述                                       |
|----------------------------|------------------------------------------|
| ClassNotFoundException     | 应用程序试图加载类时，找不到相应的类                       |
| CloneNotSupportedException | 调用 Object 类的 clone 方法，但类未实现 Cloneable 接口 |
| IllegalAccessException     | 拒绝访问一个类时抛出                               |
| InstantiationException     | 试图通过 Class 的 newInstance 创建抽象类或接口实例时抛出   |
| InterruptedException       | 一个线程被另一个线程中断时抛出                          |
| NoSuchFieldException       | 请求的变量不存在                                 |
| NoSuchMethodException      | 请求的方法不存在                                 |

**WARNING**

**必检异常（Checked Exception）** 意味着编译器会强制程序员检查并通过 try-catch 块处理它们，或者在方法头进行声明。这是 Java 与其他语言的重要区别。

## 2 Throwable 类方法

**Throwable** 是 Java 语言中所有错误和异常的超类。理解其核心方法对于异常处理至关重要。

| 序号 | 方法及说明                                                                                         |
|----|-----------------------------------------------------------------------------------------------|
| 1  | <b>public String getMessage()</b><br>返回关于异常的详细信息。这个消息在 Throwable 类的构造函数中初始化                   |
| 2  | <b>public Throwable getCause()</b><br>返回一个 Throwable 对象，代表异常原因                                |
| 3  | <b>public String toString()</b><br>返回此 Throwable 的简短描述（包含类名和 getMessage）                      |
| 4  | <b>public void printStackTrace()</b><br>将此 Throwable 及其回溯打印到标准错误流                             |
| 5  | <b>public StackTraceElement[] getStackTrace()</b><br>返回包含堆栈层次的数组。下标为 0 的元素代表栈顶，最后一个代表方法调用栈的栈底 |
| 6  | <b>public Throwable fillInStackTrace()</b><br>用当前的调用栈层次填充 Throwable 对象栈层次                     |

ThrowableMethods.java

```

1  try {
2      // 可能抛出异常的代码
3  } catch (Exception e) {
4      // 获取异常详细信息
5      System.out.println("异常信息: " + e.getMessage());
6
7      // 获取异常原因
8      Throwable cause = e.getCause();
9      if (cause != null) {
10         System.out.println("原始异常: " + cause.getMessage());
11     }
12
13     // 打印完整堆栈信息
14     e.printStackTrace();
15
16     // 获取堆栈数组进行分析
17     StackTraceElement[] stackTrace = e.getStackTrace();
18     for (StackTraceElement element : stackTrace) {
19         System.out.println(element.getClassName() + "." +
20                             element.getMethodName() + "(): " +
21                             element.getLineNumber());
22     }
23 }

```

### 3 异常类型体系

Java 的异常体系呈树形结构，所有异常都继承自 **Throwable** 类。

| 类型               | 描述                   | 代表异常                                                    |
|------------------|----------------------|---------------------------------------------------------|
| <b>Error</b>     | 程序中无法处理的错误，表示严重的系统问题 | VirtualMachineError、OutOfMemoryError、StackOverflowError |
| <b>Exception</b> | 程序本身可以捕获并且可以处理的异常    | -                                                       |

#### 3.1 Error（系统错误）

**Error** 类描述的是内部系统错误，这类错误很少发生，通常由 JVM 抛出。

| 类                    | 可能引起异常的原因                     |
|----------------------|-------------------------------|
| LinkageError         | 一个类对另一个类有依赖性，但编译前者后后者进行了不兼容修改 |
| VirtualMachineError  | Java 虚拟机崩溃或运行所必需的资源已经耗尽       |
| OutOfMemoryError     | 内存耗尽（JVM 堆内存不足）               |
| NoClassDefFoundError | 无法加载某个 Class（类路径配置错误）         |
| StackOverflowError   | 栈溢出（递归调用没有正确终止条件）             |

**CAUTION**

**Error 与 Exception 的关键区别：**

- Error 通常是 JVM 层面的问题，程序无法恢复
- Exception 是程序逻辑问题，理论上可以捕获和处理
- 但实际开发中，不建议捕获 Error，因为无法有效恢复

## 3.2 运行时异常（RuntimeException）

**运行时异常**是程序设计错误，例如错误的类型转换、访问越界数组等。

| 类                         | 可能引起异常的原因                |
|---------------------------|--------------------------|
| ArithmeticException       | 整数除以 0（注意：浮点数的算术运算不抛出异常） |
| NullPointerException      | 试图通过 null 引用变量访问对象       |
| IndexOutOfBoundsException | 数组的下标超出范围                |
| IllegalArgumentException  | 传递给方法的参数非法或不合适           |

**NOTE**

**RuntimeException 的特殊性：**

- 无需在 **throws** 子句中声明，因为无法提前预防
- 此类异常在运行期间才能发现，一旦发生程序直接终止
- 体现了“快速失败”原则：发现问题后立即暴露，而不是隐藏

RuntimeException、Error 及其子类都称为免检异常（unchecked exception），编译器不会强制检查。所有其他异常（编译时异常）都称为必检异常（checked exception）。

# 4 异常处理方式

## 4.1 声明异常（throws）

在 Java 中，方法必须声明它可能抛出的必检异常的类型，这称为**声明异常**。

ThrowsDeclaration.java

```

1 // 声明可能抛出 IOException
2 public void readFile() throws IOException {
3     // 方法实现
4 }
5
6 // 声明多个异常
7 public void process() throws IOException, SQLException {
8     // 方法实现
9 }
10
11 // 重写方法中的 throws

```

```

12 class Parent {
13     public void method() throws IOException {}
14 }
15
16 class Child extends Parent {
17     @Override
18     public void method() throws IOException {} // 可以声明相同的异常
19 }

```

**CAUTION****throws 声明规则:**

- 如果方法没有在父类中声明异常，那么就不能在子类中对其进行继承来声明异常
- 子类重写方法可以抛出父类声明异常的子类，但不能抛出新的必检异常
- 如果父类方法没有声明异常，子类重写方法也不能声明必检异常

## 4.2 抛出异常 (throw)

检测到错误的程序可以创建一个异常实例并抛出:

ThrowDemo.java

```

1 // 抛出内置异常
2 public void validate(int age) {
3     if (age < 0) {
4         throw new IllegalArgumentException("年龄不能为负数");
5     }
6 }
7
8 // 抛出自定义异常
9 public void processOrder(Order order) {
10    if (order == null) {
11        throw new NullPointerException("订单对象不能为 null");
12    }
13 }
14
15 // 直接抛出
16 throw new RuntimeException("这是一个运行时异常");

```

**TIP****throw 与 throws 的区别:**

- **throws**: 用于方法声明，表示该方法可能抛出某些异常
- **throw**: 用于实际抛出异常实例

**WARNING**

异常抛出后，try 块中 throw 之后的代码不会执行。例如:

ThrowFlow.java

```

1 try {
2     throw new Exception("test");
3     System.out.println("这段代码不会执行"); // 不可达代码
4 } catch (Exception e) {
5     // 捕获处理
6 }

```

## 5 try-catch-finally 详解

## 5.1 各部分职责

- **try 语句块**: 放置可能出现问题的代码, 尽量不要把不相关的代码放入其中
- **catch 语句块**: 放置出现问题并捕获后的处理代码, 如果 try 中没有问题发生则不会执行
- **finally 语句块**: 放置不管异常是否产生都要执行的代码, 通常用于资源释放

## 5.2 多重 catch 块

MultiCatchDemo.java

```
1 try {
2     // 可能抛出多种异常的代码
3     String input = args[0];
4     int num = Integer.parseInt(input);
5     int result = 10 / num;
6 } catch (ArrayIndexOutOfBoundsException e) {
7     // 处理数组越界
8     System.out.println("请提供命令行参数");
9 } catch (NumberFormatException e) {
10    // 处理数字格式错误
11    System.out.println("参数必须是有效数字");
12 } catch (ArithmeticException e) {
13    // 处理算术异常 (如除以零)
14    System.out.println("除数不能为零");
15 } catch (Exception e) {
16    // 处理其他所有异常 (通常放在最后)
17    System.out.println("未知错误: " + e.getMessage());
18 } finally {
19    // 资源清理代码
20    System.out.println("无论如何都会执行");
21 }
```

### CAUTION

JDK 7+ 的 **try-with-resources** 可以自动关闭资源, 代码更简洁:

TryWithResources.java

```
1 try (BufferedReader br = new BufferedReader(
2     new FileReader("file.txt"));
3     BufferedWriter bw = new BufferedWriter(
4     new FileWriter("output.txt"))) {
5     // 使用 br 和 bw 进行文件操作
6     String line;
7     while ((line = br.readLine()) != null) {
8         bw.write(line);
9         bw.newLine();
10    }
11 } catch (IOException e) {
12     e.printStackTrace();
13 }
```

## 6 finally 执行时机

finally 块的执行时机非常明确, 但有几种特殊情况需要注意。



## 6.1 基本执行规则

| 场景        | 执行顺序                                              |
|-----------|---------------------------------------------------|
| 正常执行      | try 块代码无异常执行完毕 → finally 块 → 方法继续执行后续代码           |
| 异常捕获      | try 块抛出异常 → 匹配 catch 块处理 → finally 块 → 方法继续执行后续代码 |
| 异常未捕获     | try 块抛出异常且无匹配 catch → finally 块 → 异常向上层调用栈抛出      |
| return 场景 | try 或 catch 块中 return, finally 会在 return 之前执行     |

## 6.2 return 与 finally 的交互

FinallyReturn.java

```

1 // 情况 1: try 中有 return, finally 中无 return
2 public static int test1() {
3     try {
4         return 1; // 步骤1: 保存返回值 1
5     } finally {
6         System.out.println("finally 执行"); // 步骤2: 执行 finally
7     }
8     // 步骤3: 返回保存的 1
9 }
10 // 输出: finally 执行, 返回 1
11
12 // 情况 2: try 中有 return, finally 中也有 return
13 public static int test2() {
14     try {
15         return 1;
16     } finally {
17         return 2; // 覆盖 try 中的返回值
18     }
19 }
20 // 返回 2 (这是不良实践, 会导致原始返回值丢失)
21
22 // 情况 3: finally 中修改返回值 - 基本类型
23 public static int test3() {
24     int i = 0;
25     try {
26         i = 1;
27         return i; // 保存返回值 1
28     } finally {
29         i = 2; // 修改 i 的值, 但返回的是之前保存的 1
30     }
31 }
32 // 返回 1
33
34 // 情况 4: finally 中修改返回值 - 引用类型
35 public static List<String> test4() {
36     List<String> list = new ArrayList<>();
37     try {
38         list.add("a");
39         return list; // 返回引用
40     } finally {
41         list.add("b"); // 修改引用指向的对象
42     }
43 }
44 // 返回的 list 包含 ["a", "b"]

```

### NOTE

#### 返回值保存机制:

- 基本类型: return 时会复制值, finally 中的修改不影响已保存的返回值
- 引用类型: return 时保存的是引用, finally 中修改对象内容会影响最终返回结果

### 6.3 finally 不执行的特殊情况

| 情况            | 说明                                           |
|---------------|----------------------------------------------|
| System.exit() | JVM 正常退出，finally 块不会执行                       |
| 线程被杀死         | Thread.stop()（已废弃）或 Thread.interrupt() 未正确处理 |
| 死循环/无限阻塞      | finally 块中存在无法完成的代码                          |
| 严重错误          | JVM 崩溃、内存溢出等，finally 可能无法执行                  |

**WARNING**

虽然 finally 块通常会执行，但在生产环境中应避免依赖其进行关键操作。如果 finally 中的代码抛出异常，会覆盖原始异常。

## 7 自定义异常

在实际开发中，Java 内置的异常类可能无法满足业务需求，需要自定义异常类。

### 7.1 基本自定义异常

MyException.java

```

1 // 自定义检查性异常
2 public class MyException extends Exception {
3     public MyException() {
4         super();
5     }
6
7     public MyException(String message) {
8         super(message);
9     }
10
11     public MyException(String message, Throwable cause) {
12         super(message, cause);
13     }
14 }
15
16 // 自定义运行时异常
17 public class BusinessException extends RuntimeException {
18     public BusinessException() {
19         super();
20     }
21
22     public BusinessException(String message) {
23         super(message);
24     }
25
26     public BusinessException(String message, Throwable cause) {
27         super(message, cause);
28     }
29 }
```

### 7.2 自定义异常的使用示例

CustomExceptionDemo.java

```

1 public class OrderService {
2
3     public void processOrder(Order order) throws MyException {
4         try {
5             // 验证订单
6             if (order == null) {
```

```
7         throw new MyException("订单对象不能为空");
8     }
9     if (order.getAmount() < 0) {
10         throw new MyException("订单金额不能为负数");
11     }
12
13     // 处理订单逻辑
14     orderRepository.save(order);
15
16     } catch (MyException e) {
17         // 记录日志或重新抛出
18         logger.error("订单处理失败: " + e.getMessage());
19         throw e;
20     }
21 }
22 }
23
24 // 调用示例
25 try {
26     orderService.processOrder(order);
27 } catch (MyException e) {
28     System.out.println(e.getMessage()); // 输出: 订单金额不能为负数
29     System.out.println("异常捕获结束!");
30 }
```

**TIP****自定义异常命名规范:**

- 以 Exception 结尾: 表示这是一个异常类
- 以业务名称开头: 如 UserException、OrderException、BusinessException
- 根据类型选择父类: 需要声明的使用 Exception, 不需要的使用 RuntimeException

## 8 异常处理最佳实践

### 8.1 异常处理原则

1. **不要捕获所有异常**: 只捕获可以合理处理的异常
2. **不要吞掉异常**: 至少要记录日志或重新抛出
3. **不要使用异常控制流程**: 异常用于异常情况, 不用于正常逻辑
4. **优先使用具体异常类型**: 避免使用通用的 Exception 或 Throwable
5. **释放资源时使用 finally 或 try-with-resources**

### 8.2 反模式示例

AntiPatterns.java

```
1 // 反模式 1: 捕获所有异常并忽略
2 try {
3     doSomething();
4 } catch (Exception e) {
5     // 不要这样做! 异常被完全忽略
6 }
7
8 // 反模式 2: 使用异常控制流程
9 try {
10     while (true) {
11         // 如果数组遍历完毕会抛出异常
12         list.remove(list.size() - 1);
13     }
14 } catch (IndexOutOfBoundsException e) {
```

```

15 // 退出循环
16 }
17
18 // 反模式 3: 捕获后再抛出, 丢失堆栈信息
19 try {
20     doSomething();
21 } catch (IOException e) {
22     throw new RuntimeException(e.getMessage()); // 错误: 丢失原始堆栈
23 }
24
25 // 正确做法
26 try {
27     doSomething();
28 } catch (IOException e) {
29     throw new RuntimeException(e); // 正确: 保留原始异常
30 }

```

#### NOTE

**异常链 (Exception Chaining)**: 当捕获一个异常后抛出另一个新异常时, 应该将原始异常作为新异常的原因传递, 这样可以保留完整的异常信息。

## 9 面试重点

### 9.1 checked exception 与 unchecked exception 的区别

| 区别点   | 检查性异常 (Checked)                                 | 非检查性异常 (Unchecked)                                  |
|-------|-------------------------------------------------|-----------------------------------------------------|
| 父类    | Exception (除 RuntimeException)                  | RuntimeException、Error                              |
| 编译器检查 | <b>强制检查</b> , 必须处理或声明                           | 不检查, 编译器不强制                                         |
| 处理方式  | 必须使用 try-catch 或 throws 声明                      | 可以选择处理或不处理                                          |
| 典型场景  | IOException、SQLException、ClassNotFoundException | NullPointerException、ArrayIndexOutOfBoundsException |
| 设计意图  | 程序可以恢复的异常情况                                     | 编程错误, 通常表示 bug                                      |

从 Java 设计的角度来看, checked exception 体现了“编译时安全”的思想, 要求开发者必须显式处理可预见的异常情况。

### 9.2 try-catch-finally 执行顺序

**核心结论**: finally 块总是会执行, 除非 JVM 退出。

ExecutionOrder.java

```

1 public class ExecutionOrder {
2
3     public static void main(String[] args) {
4         System.out.println(test()); // 输出: finally 2
5     }
6
7     public static int test() {
8         try {
9             System.out.println("try");
10            return 1;
11        } catch (Exception e) {
12            System.out.println("catch");
13            return 2;
14        } finally {
15            System.out.println("finally");
16            // finally 中的 return 会覆盖 try/catch 的返回值
17            return 3;
18        }
19    }
20 }

```

```

18     }
19 }
20 }

```

### WARNING

#### 面试高频问题:

- try 中有 return, finally 会不会执行? ——会, 在 return 之前执行
- try/catch/finally 都有 return, 最终返回哪个? ——finally 的 return 覆盖其他
- 如果 finally 中修改返回值, 基本类型和引用类型有何不同? ——基本类型不生效, 引用类型生效

## 9.3 throw 与 throws 的区别

| 区别点  | throw                 | throws        |
|------|-----------------------|---------------|
| 作用位置 | 方法内部使用                | 方法签名处使用       |
| 含义   | 抛出一个具体的异常对象           | 声明方法可能抛出的异常类型 |
| 数量   | 一次只能抛出一个异常            | 可以声明多个异常类型    |
| 类型   | 抛出的是 Throwable 或其子类实例 | 声明的是异常类型 (类名) |

### ThrowVsThrows.java

```

1 // throws 用于方法声明
2 public void readFile() throws IOException, SQLException {
3     // ...
4 }
5
6 // throw 用于抛出具体的异常
7 public void validate(String input) {
8     if (input == null) {
9         throw new IllegalArgumentException("输入不能为空");
10    }
11 }

```

## 9.4 finally 中 return 的问题

强烈不推荐在 finally 块中 return, 原因如下:

### FinallyReturnProblem.java

```

1 // 问题示例: 异常被吞掉
2 public boolean test() {
3     try {
4         throw new RuntimeException("原始异常");
5     } finally {
6         return true; // 原始异常被丢弃, 无法被外层捕获
7     }
8 }
9
10 // 调用方无法知道发生了什么异常
11 try {
12     obj.test(); // 这里不会抛出异常!
13 } catch (RuntimeException e) {
14     // 永远不会执行到这里
15 }

```

CAUTION

**finally 中 return 是极差的做法：**

- 会丢失 try/catch 中抛出的异常
- 违反“异常传播”原则
- 使代码难以调试和维护

## 9.5 异常处理与性能

异常处理有一定的性能开销：

| 场景         | 说明                                               |
|------------|--------------------------------------------------|
| 正常流程       | 无明显性能影响，异常处理只是注册处理器                              |
| 异常发生       | 创建异常对象（需要获取堆栈快照）、查找处理器的开销较大                      |
| StackTrace | getStackTrace() 或 printStackTrace() 会获取完整堆栈，开销较大 |
| 高频异常       | 如果异常频繁发生，性能影响显著，应考虑使用返回值或状态码                     |

异常应该用于**异常情况**，不要用于正常流程控制。如果一个异常频繁发生，说明设计可能有问题。

# Part III

## 高级特性与函数式编程

# IV

Part

并发编程





# 网络编程与高性能 IO

# VI

Part

## JVM 底层原理与性能调优

|   |            |    |
|---|------------|----|
| 6 | JVM .....  | 50 |
| 1 | 占位内容 ..... | 50 |

# Chapter 6

## JVM

---

### 1 占位内容

这里是 chap06 的初始内容。